

5.1 Contour-based techniques for image segmentation

Image segmentation is the process of **assigning a label to every pixel in an image** such that pixels with the same label share certain characteristics. The techniques addressing those problems can be classified into two major groups:

- Those producing regions with similar properties (*e.g.*, intensity, color, texture, or location in the image), addressed by classical segmentation methods and some DNN models (like SAM).
- Those segmenting regions with **similar meaning** (*e.g.*, **semantic segmentation** where pixels belonging to the same object category are assigned to the same region, or **instance segmentation** where each region represents an object, see Fig.1). This is faced by Deep NN methods.

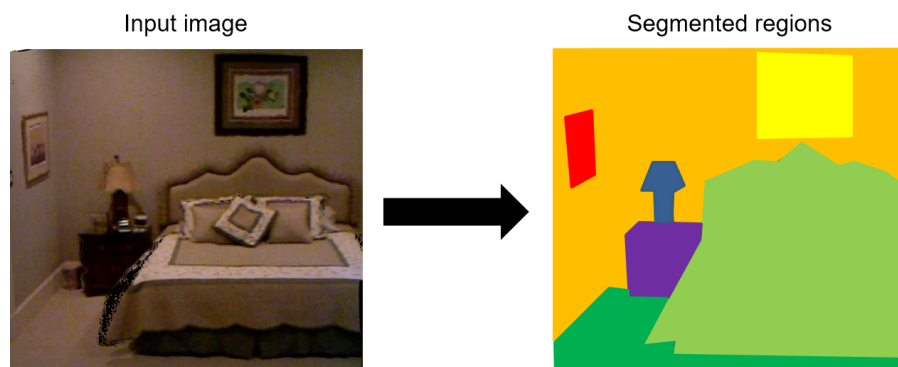


Fig. 1: Example of instance segmentation where each region corresponds to an object in the scene.

Classical methods can be further classified into:

- **Contour-based techniques**, which attempt to identify the image regions by detecting their contours. Edge pixels enclose a region with similar intensities.
- **Clustering-based techniques** that group together pixels that share some properties.

In this book we are going to experience both, starting with **contour-based techniques**, whose are based on detecting specific contours in the image (*e.g.* circles). In this context, image contours are defined as edge pixels that enclose a region.

Contour-based techniques could be roughly classified into:

- **Local techniques.** Try to segment regions by detecting closed contours, which typically enclose pixels with similar intensities.
 - LoG + zero crossing.
 - Edge following (Canny operator).
- **Global techniques.** Detect particular shapes in the image (circles, lines, etc.).
 - Hough transform.

This notebook will cover the **Hough transform** ([section 5.1.1](#)), a contour-based technique that can be used for detecting regions with an arbitrary shape in images.

Problem context - Self-driving car

A prestigious company located at PTA (The Andalusia Technology Park) is organizing a [hackathon](#) for this year in order to motivate college students to make further progress in the autonomous cars field. Computer vision students at UMA decided to take part in it, but the organizers have posed an initial basic task to guarantee that participants have expertise in image processing techniques.

This way, the company sent to students a task for **implementing a basic detector of road lane lines using OpenCV in python**. We are lucky! These are two tools that we know well ;).

Detecting lines in a lane is a fundamental task for autonomous vehicles while driving on the road. It is the building block to other path planning and control actions like breaking and steering.

So here we are! We are going to detect road lane lines using Hough transform in OpenCV.



```
In [1]: import numpy as np
import cv2
import math
import matplotlib.pyplot as plt
import matplotlib
import scipy.stats as stats

matplotlib.rcParams['figure.figsize'] = (10.0, 10.0)
images_path = './images/'
```

5.1.1 Hough transform

The **Hough Transform** is a technique for the detection of arbitrary shapes in an image. For that, such shapes must be expressed in:

- analytical form (**classic Hough**), e.g. using the mathematical representation of lines, circles, ellipses, etc., or
- in a numerical form (**generalized Hough**) where the shape is given by a table.

Since our goal is to detect lines, we will focus here on analytically expressed shapes. Specifically, a line can be represented analytically as:

$$y = ax + b$$

where a is the slope and b is the y -intercept, what is called the **explicit representation** of a line. However, this representation is problematic when representing vertical lines ($a = \infty$) or when discretizing the possible values for a , as we will see). Other option is its **parametric form**:

$$\rho = x \cos \theta + y \sin \theta$$

where ρ is the perpendicular distance from the origin $(0, 0)$ to the line, and θ is the angle formed by this perpendicular line and the horizontal axis measured in counter-clockwise (see Fig. 3). Thereby, we are going to represent lines using the pair of parameters (ρ, θ) .

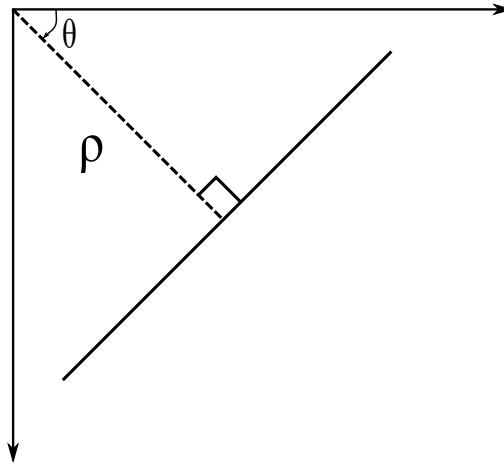


Fig. 3. Parametric representation of a line.

The Hough transform works by a voting procedure, which is carried out in a parameter space (ρ, θ in our case). This technique consists of the following steps:

1. **Build an accumulator matrix**, where rows index the possible values of ρ , and columns those for θ . For example, if the possible values for ρ are $0, 1, 2, \dots, d$ (where d is the max distance e.g. diagonal size of the image) and those for θ are $0, 1, 2, \dots, 179$, the matrix shape would be $(d, 180)$.

```
In [2]: # Define possible rho and theta values
theta_values = [0, np.pi/4, np.pi/2, 3*np.pi/4]
rho_values = [0, 1, 2, 3, 4, 5]

# Create the accumulator
acc = np.zeros([len(rho_values), len(theta_values)])
```

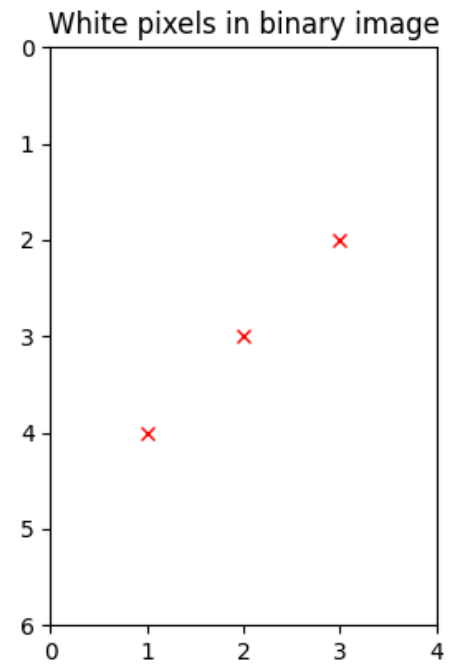
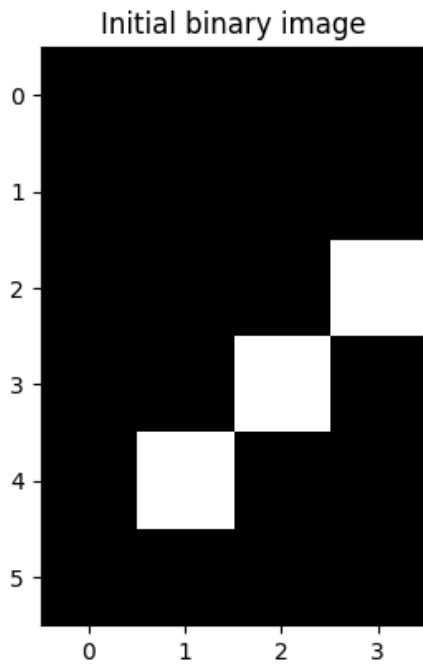
2. **Binarize the input image** to obtain pixels that are candidates to belong to the shape contours (e.g. by applying an edge detector).

```
In [3]: # Coordinates of white points in binary image
xs = np.array([1, 2, 3])
ys = np.array([4, 3, 2])

# Initial image
plt.subplot(221)
blank_image = np.zeros((6, 4, 1), np.uint8)
blank_image[ys, xs] = 255
plt.imshow(blank_image, cmap='gray');
plt.title('Initial binary image')

# Show them!
plt.subplot(222)
plt.plot(xs, ys, 'rx')
plt.axis('scaled')
```

```
axes = plt.gca()
axes.set_xlim([0,4])
axes.set_ylim([0,6])
plt.gca().invert_yaxis()
plt.title('White pixels in binary image');
```



3. For each candidate (white pixel):

A. **Evaluate:** Since the point coordinates (x, y) are known, place them in the line parametric form and iterate over the possible values of θ to obtain the values for ρ . In the previous example

$$\rho_i = x \cos \theta_i + y \sin \theta_i, \forall i \in [0, 180]$$

B. **Vote:** For every obtained pair (ρ_i, θ_i) increment by one the value of its associated cell in the accumulator.

If we do this with an accumulator with an enough resolution, we would get a sinusoid.

```
In [4]: # For each white pixel

for i in range (0,len(xs)):
    x = xs[i]
    y = ys[i]

    # Show the point voting
    subplot_index = str(32) + str(i*2+1)
    plt.subplot(int(subplot_index))
    plt.axis('scaled')
    axes = plt.gca()
    axes.set_xlim([0,4])
    axes.set_ylim([0,6])

    plt.plot(xs,ys,'rx')
```

```
plt.plot(x,y,'co')

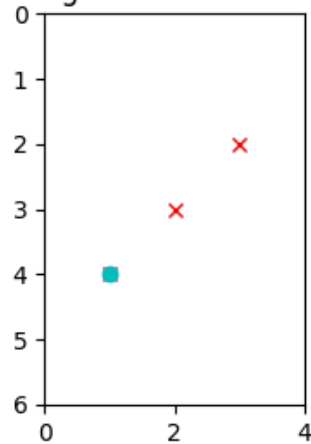
plt.title('Pixel being evaluated at iteration ' + str(i+1))
plt.gca().invert_yaxis()

# Evaluate the (x,y) coordinates for different values of theta, and
# retrieve rho
for theta_index in range(0,len(theta_values)):
    theta = theta_values[theta_index]
    rho = x*np.cos(theta) + y*np.sin(theta)
    rho = int(np.round(rho))
    # Vote!
    acc[rho][theta_index] += 1.0

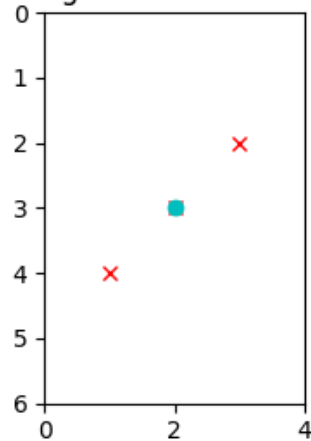
# Show the accumulator
subplot_index = str(32) + str(i*2+2)
plt.subplot(int(subplot_index))
plt.imshow(acc,cmap='gray',vmax=3);
plt.xticks([0, 1, 2, 3], ['0', 'pi/4', 'pi/2', '3pi/4'])

plt.title('Accumulator values')
```

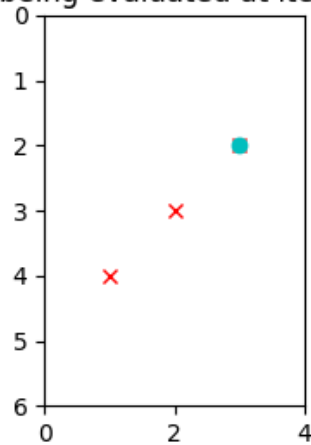
Pixel being evaluated at iteration 1



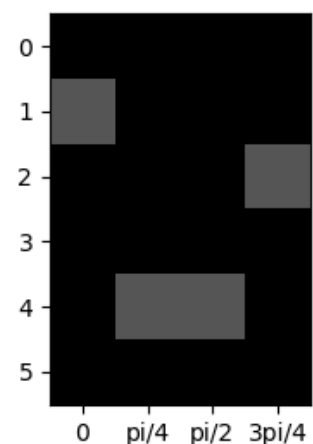
Pixel being evaluated at iteration 2



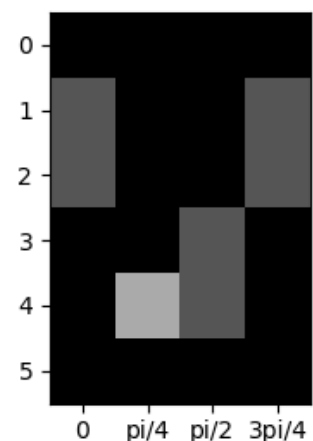
Pixel being evaluated at iteration 3



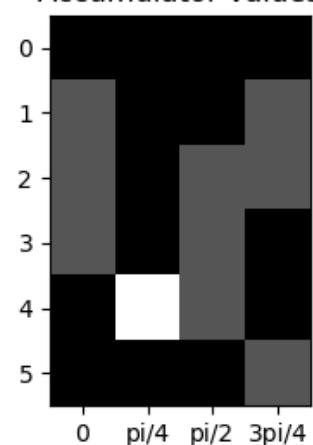
Accumulator values



Accumulator values



Accumulator values



4. Finally, **obtain the shape candidates** by setting a threshold to control how many votes needs a pair (ρ, θ) to be considered a line, and by applying local maxima in the accumulator space.

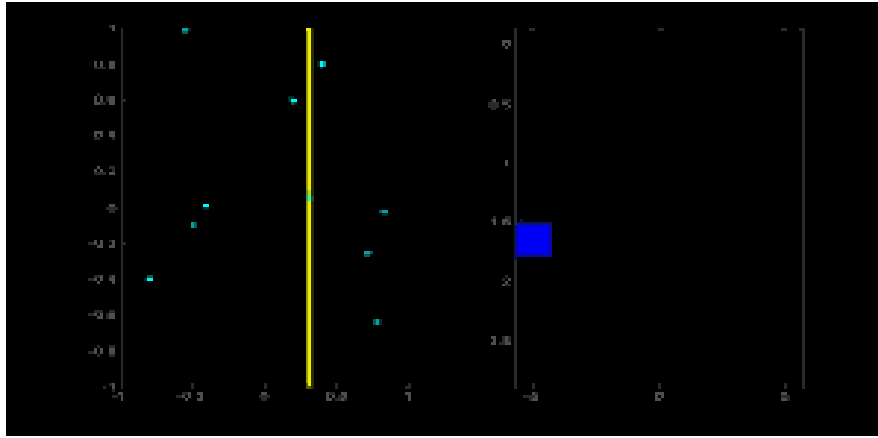


Fig. 3. Left, image space. Right, parameter space illustrating the evolution of the votes. Note that in this example θ have only 8 possible values.

The idea behind this algorithm is that when a pixel in the image space votes for all the lines that go through it in the parameter space, when a second pixel belonging to the same line votes, then the line connecting both pixels would have two votes.

OpenCV pill

OpenCV implements the method `cv2.HoughLines()` for detecting lines using the Hough transform. However, prior to its usage, and as commented in the **step 1.** of the algorithm, it is needed a binary image. For that we are going to resort to our old friend the Canny algorithm, so the detected edges will be the white pixels in the binary image.

As we now, noisy images seriously hamper the performance of computer vision techniques, and since `cv2.Canny()` does not include blurring, we provide here a method called `gaussian_smoothing()` to assist you in that task.

```
In [5]: def gaussian_smoothing(image, sigma, w_kernel):
        """ Blur and normalize input image.

        Args:
            image: Input image to be binarized
            sigma: Standard deviation of the Gaussian distribution
            w_kernel: Kernel aperture size

        Returns:
            binarized: Blurred image
        """

        # Define 1D kernel
        s=sigma
        w=w_kernel
        kernel_1D = [np.exp(-z*z/(2*s*s))/np.sqrt(2*np.pi*s*s) for z in range(-w, w+1)]
```



```
# Apply distributive property of convolution
kernel_2D = np.outer(kernel_1D, kernel_1D)

# Blur image
smoothed_img = cv2.filter2D(image, cv2.CV_8U, kernel_2D)

# Normalize to [0 254] values
smoothed_norm = np.array(image.shape)
smoothed_norm = cv2.normalize(smoothed_img, None, 0, 255, cv2.NORM_M

return smoothed_norm
```

ASSIGNMENT 1: Detecting lines with Hough

Your first task is to apply `cv2.HoughLines()` to the image `car.png`, a test image taken from the frontal camera of a car. Draw the resultant lines using `cv2.line()`.

The main inputs of `cv2.HoughLines()` are:

- *image*: binary input image
- *rho*: distance resolution of the accumulator in pixels (usually 1, it may be bigger for high resolution images)
- *theta*: angle resolution of the accumulator in radians. (usually $\frac{\pi}{180}$)
- *threshold*: only line candidates having a number of votes $>$ threshold are returned.

And it returns:

- a $(n_lines \times 2)$ array containing, in each row, the parameters of each detected line in the $[\rho, \theta]$ format.

Note that, for drawing the lines, you have to transform the resultant lines from the (ρ, θ) space to Cartesian coordinates.

Try different parameter values until you get something like this:

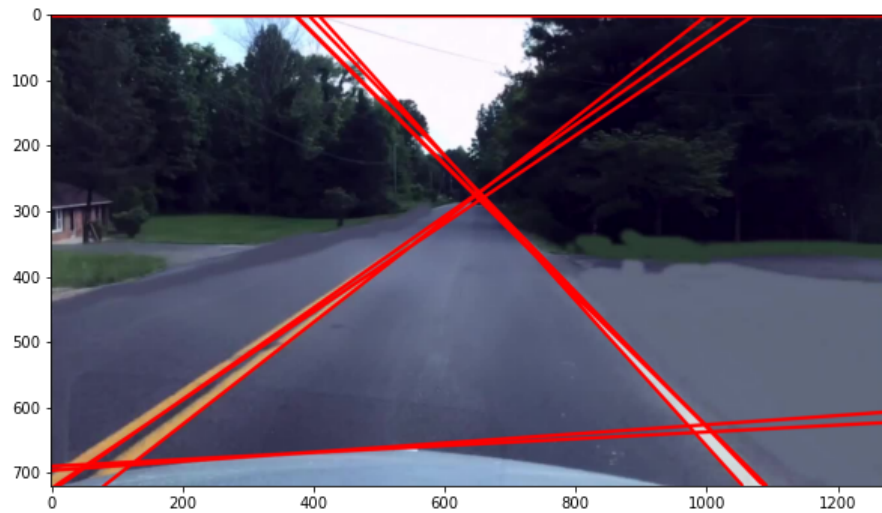


Fig. 4. Example of lines detection.

```
In [6]: # Assignment 1
# Read the image
image = cv2.imread(images_path + "car.png", -1)

# Convert to RGB and get gray image
image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
gray = cv2.cvtColor(image, cv2.COLOR_RGB2GRAY)

# Blur the gray image
gray = gaussian_smoothing(gray, 2, 5)

# Apply Canny algorithm
edges = cv2.Canny(gray, 100, 200, apertureSize = 3)

# Search for lines using Hough transform
lines = cv2.HoughLines(edges, rho=1, theta=np.pi/180, threshold=150)

# For each line
for i in range(0, len(lines)):

    # Transform from polar coordinates to cartesian coordinates
    rho = lines[i][0][0]
    theta = lines[i][0][1]

    a = math.cos(theta)
    b = math.sin(theta)

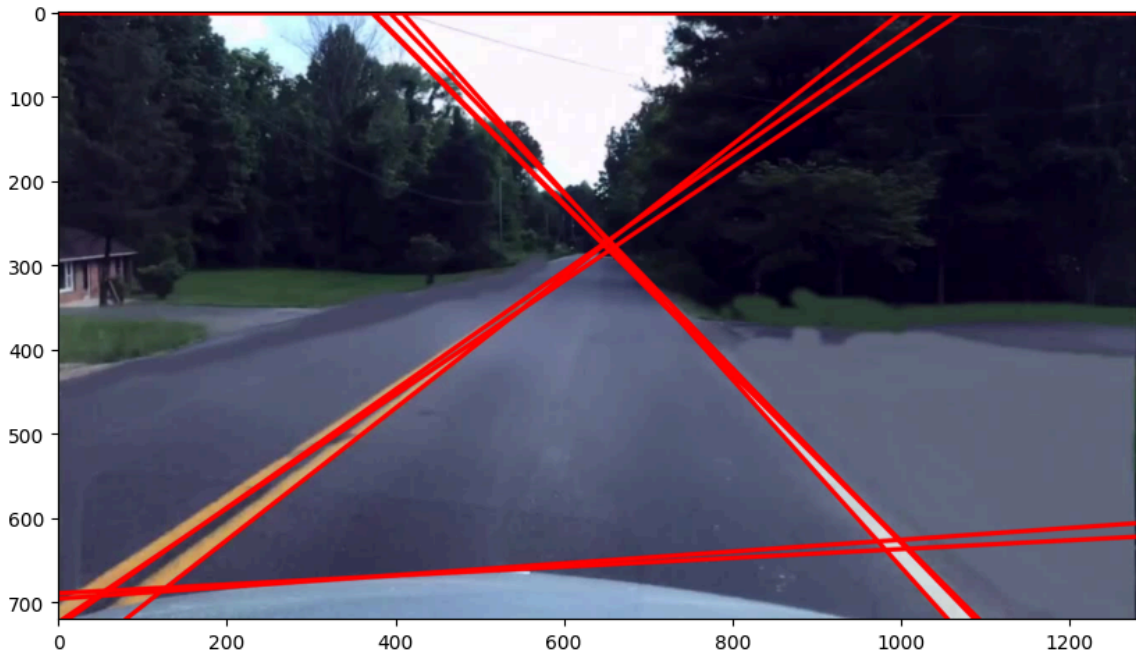
    x0 = a * rho
    y0 = b * rho

    # Get two points in that line
    x1 = int(x0 + 2000*(-b))
    y1 = int(y0 + 2000*(a))
    pt1 = (x1, y1)
    x2 = int(x0 - 2000*(-b))
    y2 = int(y0 - 2000*(a))
    pt2 = (x2, y2)
```

```
# Draw the line in the RGB image
cv2.line(image, pt1, pt2, (255,0,0), 3, cv2.LINE_AA)

# Show resultant image
plt.imshow(image)
```

Out[6]: <matplotlib.image.AxesImage at 0x1cdd30cf390>



5.1.2.1 Probabilistic Hough transform

For high-resolution images and large accumulator sizes the Hough transform may need long execution times. However, in applications like autonomous cars a fast execution is mandatory. For example, having a car moving at 100km/h covers ~ 28 meters in a second. Imagine how much lines can change in that time!

OpenCV pill

OpenCV provides with the method `cv2.HoughLinesP()` a more complex implementation of the Hough Line Transform, which is called **probabilistic Hough Transform**. This alternative does not take all the points in the binary image into account, but a random subset of them that are still enough for line detection. This also results in lower thresholds when deciding if a line exists or not.

ASSIGNMENT 2: Another option for detecting lines

Apply `cv2.HoughLinesP()` to the image `car.png` and draw the detected lines.

This function returns:

- line segments `[x1,y1,x2,y2]` instead of the line equation parameters.

For that, two additional arguments are needed:

- `minLineLength`: line segments shorter than that are rejected.
- `maxLineGap`: maximum allowed gap between points on the same line to link them.

Try different parameter values until you get something like this:

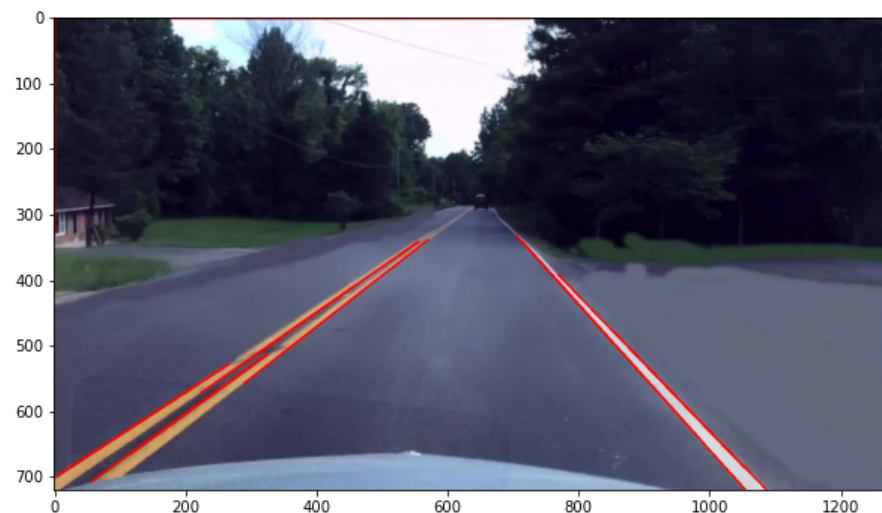


Fig. 5. Lines detection example with the probabilistic Hough Transform.

```
In [7]: # Assignment 2
# Read the image
image = cv2.imread(images_path + "car.png", -1)

# Convert to RGB and get gray image
image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
gray = cv2.cvtColor(image, cv2.COLOR_RGB2GRAY)

# Blur the gray image
gray = gaussian_smoothing(gray, 1, 2)

# Apply Canny algorithm
edges = cv2.Canny(gray, 100, 200, apertureSize = 3)

# Search for lines using probabilistic Hough transform
rho = 1
theta = np.pi/180
threshold = 150
lines = cv2.HoughLinesP(edges, rho, theta, threshold,
                        minLineLength=250, maxLineGap=15)

# For each line
for line in lines:
```

```
# Draw the line in the RGB image
x1,y1,x2,y2 = line[0]
cv2.line(image,(x1,y1),(x2,y2),(255,0,0),2)

# Show resultant image
plt.imshow(image)
```

Out[7]: <matplotlib.image.AxesImage at 0x1cdd3493ed0>



Thinking about it (1)

Now that you have played with the Hough Transform, **answer the following questions:**

- In the first assignment, we obtained an image with a number of red lines drawn on it. However, these lines go over pixels in the image that do not contain lines! (e.g. belonging to the sky). What could we do to fix this, that is, obtain the pixels belonging to lines in the image?

The standard Hough Transform doesn't detect line segments, it detects the parameters of an infinite mathematical line.

- Without restrictions regarding execution time, which method would you use, Hough Transform or its probabilistic counterpart?

Probabilistic Hough Transform because it provides the actual start and end points of line segments.

- Could there be a cell in the accumulator with a value higher than the number of white pixels in the binary image? why?

No, Since each white pixel can only cast one vote for any given line, the absolute maximum number of votes a single cell can accumulate is the

total number of white pixels in the image.

- Which should be the maximum value for ρ in the accumulator? Why?

The maximum value for rho should be the length of the diagonal of the image because rho is defined as the perpendicular distance from the origin to a line. Any line with a rho value greater than the diagonal length would be too far from the origin to intersect with any pixel in the image

OPTIONAL

Think about any other application where the finding of straight lines could be useful. Get some images related to said application and detect lines with the Hough transform!

END OF OPTIONAL PART

AI Appendix

A.1 How I used AI

- **Model/tool:**

Gemini-2.5Pro

- **What I asked AI to help with (bullets):**

Your answer here!

- **Best prompt I used (paste):**

Your answer here!

- **What I kept vs. changed from the AI output:**

Your answer here!

A.2 AI review

Ask one generative AI to review your memo using this prompt:

Act as a technical reviewer of this computer vision lab notebook. Read my memo and return exactly 5 bullet points:

- two on theory (key ideas),
- two on practice (relevant programming concepts of python used, not particular functions or variables),
- one on how to improve the code (most critical aspect: limitations, parameter choices, runtime/robustness,

possible bugs).

Keep each bullet ≤ 15 words and make them specific to my work.

Paste its **5** items here:

- **(Theory 1)** AI claim:
The Hough Transform uses an accumulator to vote in the (rho, theta) parameter space.
- **(Theory 2)** AI claim:
The notebook contrasts the full transform with the faster, subset-based Probabilistic Hough.
- **(Practice 1)** AI claim:
Image processing pipelines (blurring, edge detection) are essential prerequisites for Hough detection.
- **(Practice 2)** AI claim:
Requires coordinate system conversion from parametric polar space (rho, theta) to Cartesian for drawing
- **(Improvement)** AI claim:
Hard-coded Canny and Hough thresholds limit robustness to varying road or lighting conditions.

Finally, **do you agree with them?, would you have selected others?**

Your answer here!

Conclusion

Terrific work! In the road lane lines detection context you have learned:

- to detect shapes in images using Hough transform .

Also, you obtained some knowledge about:

- self-driving cars and computer vision, and
- lane line detection for autonomous cars.

See you in the next one! Keep learning!